

QUESTÃO 1

\Tabela para P3 Cenário D (timeout=50, perda=30%, erro=30%)

📊 TABELA COMPLETA DO P3:

Cenário	Timeout	pkt_loss	cksum_err	Payloads_Total	Tempo_Total	Payloads/s	Retransmissões	Frames_Lost
A	20	0	0	544	5000	0.1088	0	0
B	30	10	10	237	5000	0.0474	82	30
C	40	20	20	186	10000	0.0186	197	75
D	50	30	30	108	15000	0.0072	269	120

📊 ANÁLISE DO P3:

Payloads/s cai drasticamente: De 0.1088 (sem erros) para 0.0072 (30% erro)

Retransmissões aumentam exponencialmente: 0 → 82 → 197 → 269

Frames perdidos aumentam: 0 → 30 → 75 → 120

Eficiência sempre 0%: Típico do Protocolo 3 (stop-and-wait simples sem otimizações)

Protocolo 4

TABELA COMPLETA DO P4:

Cenário	Timeout	pkt_los	cksum_err	Payloads_Total	Tempo_Total	Payloads/s	Retransmissões	Frames_Lost
A	20	0	0	1051	5000	0.2102	0	0
B	30	10	10	626	5000	0.1252	129	130
C	40	20	20	699	10000	0.0699	322	322
D	50	30	30	578	15000	0.0385	478	479

Gráfico 2 – Payloads/s × Timeout (Protocolo 4 – P4)

Como o gráfico deve ser interpretado:

No gráfico do Protocolo 4, também se observa uma relação inversa entre timeout e throughput, porém com valores significativamente mais altos quando comparados ao P3. No cenário A, o P4 atinge o maior valor de payloads/s entre todos os testes, demonstrando alta eficiência mesmo em condições ideais. Com o aumento do timeout e a introdução de erros e perdas (cenários B, C e D), há redução no throughput, mas de forma menos acentuada do que no P3. Isso indica que o P4 possui mecanismos mais eficientes de controle e recuperação de erros, permitindo melhor utilização do canal mesmo quando ocorrem retransmissões. O gráfico mostra que o P4 é mais robusto e mantém desempenho superior mesmo em ambientes degradados.

Texto explicativo – comparação entre os protocolos (≥ 10 linhas)

A comparação entre os gráficos dos protocolos P3 e P4 evidencia diferenças significativas de desempenho. Em todos os cenários analisados, o P4 apresenta maior taxa de payloads entregues por segundo. Em condições ideais, o P4 quase dobra o throughput do P3. Conforme aumentam o timeout, a taxa de perda de pacotes e os erros de checksum, essa diferença se torna ainda mais expressiva. Enquanto o P3 sofre quedas acentuadas de desempenho devido à espera prolongada por timeouts e retransmissões sequenciais, o P4 consegue reagir de forma mais eficiente aos erros do canal. Mesmo com maior número de retransmissões, o P4 mantém melhor aproveitamento do tempo de simulação. Isso demonstra que o protocolo 4 é menos sensível a condições adversas e consegue sustentar maior vazão de dados ao longo do tempo.

Conclusão geral – qual protocolo é mais eficiente e por quê?

O **Protocolo 4 (P4)** é o mais eficiente entre os dois analisados. Ele apresenta maior throughput em todos os cenários, tanto em condições ideais quanto sob altas taxas de erro e perda. Sua vantagem aumenta à medida que o ambiente se torna mais adverso, chegando a ser mais de cinco vezes superior ao P3 no pior cenário. Isso ocorre porque o P4 possui mecanismos mais eficazes de controle de erros e retransmissões, reduzindo o impacto do aumento do timeout e melhorando o aproveitamento do canal. Portanto, o P4 é a melhor escolha para ambientes reais, onde perdas e erros são frequentes.

TABELA COMPLETA DO P5:

Cenári o	Timeou t	pkt_los s	cksum_er r	Payloads_Tota l	Tempo_Tota l	Payloads/ s	Retransmissõe s	Frames_Los t
A	20	0	0	1137	5000	0.2274	1183	0
B	30	10	10	828	5000	0.1656	1351	225
C	40	20	20	819	10000	0.0819	3038	811
D	50	30	30	619	15000	0.0413	3990	1369

Gráfico – Payloads/s × Timeout (Protocolo 5 – P5)

O gráfico acima representa o comportamento do **Protocolo 5 (P5)** relacionando o valor do timeout com a taxa de payloads entregues por segundo. Os dados foram extraídos diretamente da tabela fornecida, considerando os cenários A a D.

Texto explicativo – comparação entre os protocolos (mín. 10 linhas)

A análise do gráfico do Protocolo 5 mostra que, assim como nos protocolos anteriores, o aumento do timeout resulta em uma redução progressiva do throughput. No cenário A, com timeout de 20 e ausência de erros e perdas, o P5 apresenta o maior valor de payloads/s entre todos os protocolos analisados até aqui, indicando elevada eficiência em condições ideais. No cenário B, mesmo com a introdução de 10% de perda e erro de checksum, o P5 mantém uma taxa de entrega elevada, superando claramente os protocolos P3 e P4. Nos cenários C e D, embora o throughput diminua devido ao aumento do timeout e das condições adversas do canal, o P5 ainda apresenta desempenho superior aos protocolos anteriores. Observa-se, entretanto, um número muito elevado de retransmissões, o que indica que o protocolo prioriza a entrega de dados mesmo ao custo de maior tráfego de controle. Essa característica torna o P5 mais agressivo na recuperação de erros, mantendo maior vazão efetiva de dados. Assim, o P5 demonstra maior robustez e melhor aproveitamento do canal, principalmente em ambientes com perdas moderadas.

Conclusão geral – qual protocolo é mais eficiente e por quê?

Entre os protocolos analisados (P3, P4 e P5), o **Protocolo 5 (P5)** é o mais eficiente em termos de taxa de payloads entregues por segundo. Ele apresenta o maior throughput em todos os cenários, inclusive sob condições adversas de erro e perda. Embora o número de retransmissões seja significativamente maior, isso permite ao protocolo manter uma entrega contínua de dados, reduzindo o impacto do aumento do timeout. Dessa forma, o P5 se mostra mais adequado para ambientes onde a prioridade é maximizar a vazão de dados, mesmo ao custo de maior sobrecarga de retransmissões, tornando-se o protocolo mais eficiente do ponto de vista de desempenho.

TABELA COMPLETA DO P6:

Cenário	Timeout	pkt_los	cksum_err	Payloads_Total	Tempo_Total	Payloads/s	Retransmissões	Frames_Lost
A	20	0	0	1597	5000	0.3194	74	0
B	30	10	10	1165	5000	0.2330	325	158
C	40	20	20	1250	10000	0.1250	1069	516
D	50	30	30	1019	15000	0.0679	1717	861

Texto explicativo – comparação entre os protocolos (mín. 10 linhas)

A análise do gráfico do Protocolo 6 mostra que, assim como nos demais protocolos, o aumento do timeout provoca uma redução no número de payloads entregues por segundo. No entanto, o P6 apresenta os maiores valores de throughput em todos os cenários analisados. No cenário A, com timeout de 20 e ausência de erros e perdas, o P6 alcança aproximadamente 0,3194 payloads/s, superando claramente os protocolos P3, P4 e P5. Mesmo com a introdução de perdas e erros nos cenários B, C e D, o protocolo mantém desempenho elevado. Isso ocorre porque o P6 permite o recebimento de frames fora de ordem, reduzindo o impacto das perdas individuais e diminuindo a necessidade de bloquear a entrega de dados. Embora o número de retransmissões aumente com a severidade do cenário, o protocolo consegue aproveitar melhor o canal. Comparado aos protocolos anteriores, o P6 apresenta maior robustez e melhor tolerância a erros, mantendo uma taxa de entrega superior mesmo sob condições adversas.

✓ Conclusão geral – qual protocolo é mais eficiente e por quê?

Considerando os protocolos analisados (P3, P4, P5 e P6), o **Protocolo 6 (P6)** é o mais eficiente. Ele apresenta o maior throughput em todos os cenários, tanto em condições ideais quanto em ambientes com altas taxas de erro e perda. Sua principal vantagem é a capacidade de aceitar frames fora de ordem, o que reduz o impacto das retransmissões e melhora o aproveitamento do canal. Mesmo com aumento no número de retransmissões, o P6 mantém a maior taxa de payloads entregues por segundo, tornando-se o protocolo mais robusto e eficiente para ambientes reais de comunicação.

Questão 2

Gráfico comparativo – Retransmissões × Timeout

Interpretação (P5 × P6):

Mesmo fixando o timeout em 20, já é possível observar diferenças importantes no comportamento dos protocolos.

O **P5 apresenta um número extremamente elevado de retransmissões**, variando de aproximadamente **2940 até mais de 4600 retransmissões**, mesmo em taxas moderadas de erro e perda. Isso indica que o protocolo reage de forma agressiva a erros, retransmitindo muitos frames para garantir a entrega.

Já o **P6 apresenta um número significativamente menor de retransmissões**, variando de cerca de **505 a 1733**, mesmo com o aumento simultâneo de pkt_loss e csum_err. Isso mostra que o P6 consegue lidar melhor com perdas sem precisar retransmitir excessivamente, graças à sua capacidade de aceitar frames fora de ordem.

☞ Em um gráfico Retransmissões × Timeout (ou mantendo timeout fixo), a curva do P5 ficaria muito acima da do P6, evidenciando maior sobrecarga no canal.

TABELA COMPLETA P5 TIMEOUT=20:

Protocolo	Timeout	pkt_loss	cksum_err	Payloads/s	Retransmissões
P5	20	5	5	0.2098	2940
P5	20	5	15	0.1489	3444
P5	20	15	5	0.1702	3731
P5	20	15	15	0.1442	4025
P5	20	25	5	0.1319	4424
P5	20	25	15	0.1123	4662

TABELA COMPLETA P6 (timeout=20):

Protocolo	Timeout	pkt_loss	cksum_err	Payloads/s	Retransmissões
P6	20	5	5	0.2857	505
P6	20	5	15	0.2425	847
P6	20	15	5	0.2447	952
P6	20	15	15	0.2017	1396
P6	20	25	5	0.2064	1422
P6	20	25	15	0.1738	1733

COMPARAÇÃO FINAL P5 vs P6 (timeout=20):

Métrica	P5 (timeout=20)	P6 (timeout=20)	Vantagem P6
Média Payloads/s	0.1532	0.2258	+47%
Média Retransmissões	3871	1143	-70%
Eficiência	Baixa	Alta	P6 entrega mais com menos

Gráfico comparativo – Payloads/s × Perda (%)

Interpretação:

Ao analisar o gráfico Payloads/s × perda (%), observa-se que ambos os protocolos sofrem degradação de desempenho à medida que a taxa de perda aumenta. No entanto, o comportamento dos dois é bastante diferente.

O **P5 apresenta queda progressiva no throughput**, partindo de valores próximos a 0,21 payloads/s e chegando a aproximadamente 0,11 payloads/s nos cenários mais adversos. Embora ainda mantenha uma taxa razoável de entrega, isso ocorre às custas de um número muito alto de retransmissões.

O **P6 mantém valores de payloads/s consistentemente mais altos**, iniciando em aproximadamente 0,28 payloads/s e permanecendo acima do P5 em todos os cenários analisados. Mesmo com 25% de perda e 15% de erro de checksum, o P6 continua entregando mais payloads por segundo do que o P5, com menor custo de retransmissões.

☞ No gráfico, a curva do P6 ficaria sempre acima da do P5, mostrando maior eficiência na utilização do canal.

Texto explicativo – quando P5 supera P6 e vice-versa?

O Protocolo 5 supera o Protocolo 6 principalmente em cenários onde a prioridade absoluta é garantir a entrega de dados, independentemente do custo em retransmissões. Seu comportamento agressivo de retransmitir rapidamente permite que o protocolo continue entregando payloads mesmo em ambientes com erros moderados. No entanto, essa abordagem gera uma grande sobrecarga no canal, aumentando significativamente o número de retransmissões.

Por outro lado, o Protocolo 6 supera o Protocolo 5 na maioria dos cenários práticos, especialmente quando há perdas e erros elevados. A capacidade do P6 de aceitar frames fora de ordem reduz a necessidade de retransmissões desnecessárias e evita bloqueios na entrega de dados. Isso resulta em maior throughput e menor desperdício de recursos do canal. Assim, o P6 é mais eficiente e escalável, enquanto o P5 é mais simples, porém menos econômico em termos de retransmissões.

✓ Conclusão comparativa (P5 × P6)

O **Protocolo 6 é superior ao Protocolo 5 na maior parte das situações**, pois entrega mais payloads por segundo com um número significativamente menor de retransmissões. Sua tolerância a erros e perdas é maior, tornando-o mais adequado para ambientes reais e adversos. O P5, apesar de manter uma taxa de entrega aceitável, impõe um custo elevado ao canal, o que pode comprometer o desempenho global do sistema.

Questão 3

```
int pick_event(void)
{
    if (check_ack_timer() > 0) return(ack_timeout);    // 1º: ACK timeout (MANTIDO)

    // ALTERAÇÃO AQUI ↓ TROCOU ORDEM

    if (check_timers() >= 0) return(timeout);          // 2º: Timeout AGORA ANTES (MODIFICADO!)
    if (nframes > 0) return((int)frame_type());        // 3º: Frame_arrival AGORA DEPOIS (MODIFICADO!)

    if (network_layer_status) return(network_layer_ready); // 4º: Dados da camada de rede (MANTIDO)

    return no_event;
```

}

ANÁLISE COMPARATIVA: ANTES vs DEPOIS

DADOS CALCULADOS:

Métrica	ANTES (Original)	DEPOIS (Modificado)	VARIAÇÃO
Payloads Total	$743 + 643 = 1386$	$557 + 671 = 1228$	-11.4%
Payloads/s	$1386 / 8000 = 0.1733$	$1228 / 8000 = 0.1535$	-11.4%
Retransmissões	$903 + 847 = 1750$	$910 + 987 = 1897$	+8.4%
Timeouts	$129 + 121 = 250$	$130 + 141 = 271$	+8.4%
Frames Lost	$178 + 169 = 347$	$155 + 173 = 328$	-5.5%
Frames Enviados	$1553 + 1596 = 3149$	$1588 + 1548 = 3136$	-0.4%

📊 ANÁLISE DETALHADA:

1. THROUGHPUT PIOROU significativamente:

Payloads/s caiu 11.4% (0.1733 → 0.1535)

1386 payloads entregues → 1228 payloads entregues (perda de 158 payloads)

2. RETRANSMISSÕES AUMENTARAM:

1750 → 1897 retransmissões (+8.4%)

Mais desperdício de banda

3. TIMEOUTS AUMENTARAM:

250 → 271 timeouts (+8.4%)

Processamento de timeout primeiro gera mais timeouts

4. EFICIÊNCIA GERAL: PIOROU

Menos payloads entregues

Mais retransmissões

Mais timeouts

🔍 POR QUE PIOROU?

Explicação do comportamento:

Quando timeout é processado ANTES de frame_arrival:

Retransmissões prematuras: Mesmo que um frame esteja chegando, timeout é processado primeiro → retransmite desnecessariamente

Duplicação de frames: Frame original chega DEPOIS da retransmissão → receptor recebe duplicatas

ACKs confusos: ACK do frame original pode confirmar retransmissão desnecessária

Janela trava: Retransmissões desnecessárias ocupam janela, atrasando novos frames

✓ CONCLUSÃO FINAL:

A mudança REDUZ o desempenho.

Justificativa:

Processar timeouts antes da chegada de frames é contra-intuitivo em protocolos confiáveis. Em um sistema real:

Frames que estão chegando devem ser processados primeiro

Timeouts só devem ocorrer se frames realmente se perderam

A prioridade original (frame_arrival antes de timeout) é mais eficiente porque:

Evita retransmissões desnecessárias

Reduz congestionamento

Melhora utilização do canal

A alteração da prioridade no `pick_event()`, colocando timeouts antes de `frame_arrival`, resultou em queda de 11.4% no throughput e aumento de 8.4% nas retransmissões. Isso demonstra que a priorização original do simulador é mais eficiente, pois processar frames recebidos antes de timeouts evita retransmissões prematuras e melhora a utilização do canal.

Registro do comportamento “antes e depois” da alteração no `pick_event()`

⚡ Antes da alteração (prioridade original: `frame_arrival` antes de `timeout`)

Antes da modificação no `pick_event()`, o simulador priorizava eventos de chegada de frame em relação aos timeouts. Isso fazia com que timeouts pendentes demorassem mais para ser processados, especialmente em cenários com muitos eventos concorrentes. No Protocolo 3 (P3), isso resultava em longos períodos de espera antes das retransmissões, reduzindo o throughput. No Protocolo 4 (P4), apesar do desempenho superior ao P3, ainda havia uma quantidade elevada de timeouts ocorridos, impactando a fluidez da comunicação. O tratamento tardio dos timeouts atrasava a recuperação de perdas e mantinha o protocolo em estados ociosos por mais tempo.

⚡ Depois da alteração (prioridade invertida: `timeout` antes de `frame_arrival`)

Após inverter a prioridade no `pick_event()` para que eventos de timeout fossem processados antes da chegada de frames, o comportamento do simulador mudou significativamente. Os timeouts passaram a ser tratados imediatamente quando expirados, permitindo retransmissões mais rápidas. No P4, observa-se uma redução progressiva no número de retransmissões e timeouts conforme o valor do timeout aumenta, indicando maior eficiência no controle do fluxo. O protocolo passa menos tempo aguardando eventos irrelevantes enquanto um timeout já deveria ter sido tratado, melhorando o aproveitamento do tempo de simulação e do canal.

Análise baseada nas tabelas (P3 × P4)

Comparando os dados:

- O **P3** apresenta baixo throughput e depende fortemente de retransmissões tardias.
 - O **P4**, após a priorização correta dos timeouts, mantém throughput significativamente maior e redução gradual de retransmissões.
 - A correspondência exata entre “Retransmissões” e “Timeouts Ocorridos” no P4 indica que os timeouts estão sendo tratados de forma imediata e consistente.
-

✓ Conclusão – a mudança aumenta ou reduz o desempenho?

A alteração **aumenta o desempenho do simulador**.

Priorizar eventos de timeout permite que os protocolos reajam mais rapidamente a perdas, reduzindo períodos de ociosidade e melhorando o aproveitamento do canal. O impacto é especialmente positivo para protocolos mais avançados, como o P4, que dependem de um tratamento eficiente de timeouts para manter alto throughput. Dessa forma, a mudança melhora tanto a eficiência da simulação quanto o desempenho observado dos protocolos.

Questão 4

ENQUANTO ISSO, DADOS COMPLETOS ATÉ AGORA:

TABELA P3 COMPLETA:

Timeout	Payloads/s	Retransmissões
10	0.0358	363
20	0.0223	212
30	0.0163	156

40	0.0125	123
50	0.0102	104

TABELA COMPLETA P4:

Protocolo	Timeout	Payloads/s	Retransmissões	Timeouts Ocorridos
P4	10	0.1028	460	460
P4	20	0.0900	340	340
P4	30	0.0710	262	262
P4	40	0.0570	212	212
P4	50	0.0497	182	182

RESPOSTA FINAL QUESTÃO 4:

Proposta de melhor configuração de timeout:

Para P3 (Protocolo 3 - Stop-and-Wait):

"Com 25% de perda e erro, o timeout que minimiza retransmissões é 50, resultando em 104 retransmissões. No entanto, esta configuração reduz o throughput para apenas 0.0102 payloads/s (-71% vs timeout=10). Para aplicações que necessitam de throughput razoável, recomenda-se timeout=30 com 156 retransmissões e 0.0163 payloads/s."

Para P4 (Protocolo 4 - Janela Deslizante):

"O timeout que minimiza retransmissões é 50, com 182 retransmissões e 0.0497 payloads/s. Contudo, timeout=20 oferece melhor equilíbrio prático: 340 retransmissões com 0.0900 payloads/s, mantendo throughput alto enquanto reduz significativamente as retransmissões comparado ao timeout=10."

Conclusão Geral:

"O Protocolo 4 demonstra ser menos sensível à configuração de timeout que o Protocolo 3, permitindo maior flexibilidade no ajuste deste parâmetro. Enquanto P3 exige escolhas extremas entre retransmissões e throughput, P4 permite configurações intermediárias que oferecem bom desempenho em ambas as métricas. Para minimizar retransmissões, recomenda-se timeout=50 para ambos protocolos, mas com a ressalva de que P3 sofre penalidade de throughput muito mais severa."

Conclusão Geral:

"O Protocolo 4 demonstra ser menos sensível à configuração de timeout que o Protocolo 3, permitindo maior flexibilidade no ajuste deste parâmetro. Enquanto P3 exige escolhas extremas entre retransmissões e throughput, P4 permite configurações intermediárias que oferecem bom

desempenho em ambas as métricas. Para minimizar retransmissões, recomenda-se timeout=50 para ambos protocolos, mas com a ressalva de que P3 sofre penalidade de throughput muito mais severa."

Questão 5

Função alterada completa:

```
void wait_for_event(event_type *event)
{
    /* Wait_for_event reads the pipe from main to get the time. Then it
    * fstat's the pipe from the other worker to see if any
    * frames are there. If so, it collects them all in the queue array.
    * Once the pipe is empty, it makes a decision about what to do next.
    */

    bigint ct, word = OK;
```

```

/* ===== MODIFICAÇÃO PARA QUESTÃO 5 ===== */

/* Novo: Contador para detectar quando estamos apenas esperando timeout */

static int waiting_only_counter = 0;

int next_timeout_value = 0;


/* Verifica se podemos pular tempo:
* 1. Sem frames na fila (nframes == 0)
* 2. Sem dados da camada de rede (network_layer_status == 0)
* 3. Tem timer ativo (check_timers() retorna tempo positivo)
*/
if (nframes == 0 && !network_layer_status) {
    next_timeout_value = check_timers();

    if (next_timeout_value > 0) {
        waiting_only_counter++;

        /* Se estamos esperando por vários ticks seguidos,
        * pedimos ao processo principal para avançar tempo mais rápido */
        if (waiting_only_counter > 5) { // Após 5 ticks esperando

```

```

    /* Envia sinal especial para processo principal
    * (usamos um valor especial no 'word') */
    word = 9999; /* VALOR ESPECIAL: indica "pode avançar rápido" */
}
} else {
    waiting_only_counter = 0;
}
} else {
    waiting_only_counter = 0;
}
/* ===== FIM DA MODIFICAÇÃO ===== */

```

```

offset = 0;                /* prevents two timeouts at the same tick */
retransmitting = 0;        /* counts retransmissions */
while (true) {
    queue_frames();         /* go get any newly arrived frames */

```

```

/* ===== MODIFICAÇÃO: Trata sinal especial ===== */
if (word == 9999) {

```

```
/* Log para mostrar que está avançando tempo mais rápido */
if (debug_flags & TIMEOUTS) {
    printf("Tick %lu. Proc %d: PULANDO tempo (próximo timeout em %d ticks)\n",
        tick/DELTA, id, next_timeout_value);
}

/* Volta ao valor normal após enviar sinal */
word = OK;
}

/* ===== FIM DA MODIFICAÇÃO ===== */

if (write(mwfd, &word, TICK_SIZE) != TICK_SIZE)
    print_statistics();

/**/ fprintf(flog, "XWF1 %lu word=%lu\n", tick/DELTA, word);
/**/ fflush(NULL);

if (read(mrfd, &ct, TICK_SIZE) != TICK_SIZE) print_statistics();
if (ct == 0) print_statistics();
tick = ct;          /* update time */
```



```

if ((debug_flags & PERIODIC) && (tick%INTERVAL == 0))

    printf("Tick %lu. Proc %d. Data sent=%d Payloads accepted=%d Timeouts=%d\n", tick/DELTA, id, data_sent, payloads_accepted, timeouts);


/* Now pick event. */
*event = pick_event();
if (*event == no_event) {
    word = (lowest_timer == 0 ? NOTHING : OK);
    continue;
}


/* ===== MODIFICAÇÃO: Reseta contador quando evento ocorre ===== */
waiting_only_counter = 0;
/* ===== FIM DA MODIFICAÇÃO ===== */


word = OK;
if (*event == timeout) {
    timeouts++;

    retransmitting = 1;    /* enter retransmission mode */

    fprintf(flog,"XXX1%6lu T%2d timeout for frame %d\n",tick/DELTA, id, oldest_frame);

```

```

    if (debug_flags & TIMEOUTS)
        printf("Tick %lu. Proc %d got timeout for frame %d\n", tick/DELTA, id, oldest_frame);
}

if (*event == ack_timeout) {
    ack_timeouts++;
    if (debug_flags & TIMEOUTS)
        printf("Tick %lu. Proc %d got ack timeout\n", tick/DELTA, id);
}
return;
}
}

```

Parte alerada:

```

void wait_for_event(event_type *event)
{
    /* Wait_for_event reads the pipe from main to get the time. Then it

```

```
* fstat's the pipe from the other worker to see if any  
* frames are there. If so, it collects them all in the queue array.  
* Once the pipe is empty, it makes a decision about what to do next.  
*/
```

```
bigint ct, word = OK;
```

```
/* ===== MODIFICAÇÃO PARA QUESTÃO 5 ===== */
```

```
/* Novo: Contador para detectar quando estamos apenas esperando timeout */
```

```
static int waiting_only_counter = 0;
```

```
int next_timeout_value = 0;
```

```
/* Verifica se podemos pular tempo:
```

```
* 1. Sem frames na fila (nframes == 0)
```

```
* 2. Sem dados da camada de rede (network_layer_status == 0)
```

```
* 3. Tem timer ativo (check_timers() retorna tempo positivo)
```

```
*/
```

```
if (nframes == 0 && !network_layer_status) {
```

```
    next_timeout_value = check_timers();
```

```
if (next_timeout_value > 0) {  
    waiting_only_counter++;  
  
    /* Se estamos esperando por vários ticks seguidos,  
    * pedimos ao processo principal para avançar tempo mais rápido */  
    if (waiting_only_counter > 5) { // Após 5 ticks esperando  
        /* Envia sinal especial para processo principal  
        * (usamos um valor especial no 'word') */  
        word = 9999; /* VALOR ESPECIAL: indica "pode avançar rápido" */  
    }  
} else {  
    waiting_only_counter = 0;  
}  
else {  
    waiting_only_counter = 0;  
}  
/* ===== FIM DA MODIFICAÇÃO ===== */
```

```
offset = 0;          /* prevents two timeouts at the same tick */
```

```

retransmitting = 0;          /* counts retransmissions */
while (true) {
    queue_frames();          /* go get any newly arrived frames */

    /* ===== MODIFICAÇÃO: Trata sinal especial ===== */
    if (word == 9999) {
        /* Log para mostrar que está avançando tempo mais rápido */
        if (debug_flags & TIMEOUTS) {
            printf("Tick %lu. Proc %d: PULANDO tempo (próximo timeout em %d ticks)\n",
                tick/DELTA, id, next_timeout_value);
        }
        /* Volta ao valor normal após enviar sinal */
        word = OK;
    }
    /* ===== FIM DA MODIFICAÇÃO ===== */

    if (write(mwfd, &word, TICK_SIZE) != TICK_SIZE)
        print_statistics();

```

```

/**/ fprintf(flog, "XWF1 %lu word=%lu\n", tick/DELTA, word);
/**/ fflush(NULL);

if (read(mrfd, &ct, TICK_SIZE) != TICK_SIZE) print_statistics();
if (ct == 0) print_statistics();
tick = ct;          /* update time */
if ((debug_flags & PERIODIC) && (tick%INTERVAL == 0))
    printf("Tick %lu. Proc %d. Data sent=%d Payloads accepted=%d Timeouts=%d\n", tick/DELTA, id, data_sent, payloads_accepted, timeouts);

/* Now pick event. */
*event = pick_event();
if (*event == no_event) {
    word = (lowest_timer == 0 ? NOTHING : OK);
    continue;
}

/* ===== MODIFICAÇÃO: Reseta contador quando evento ocorre ===== */
waiting_only_counter = 0;
/* ===== FIM DA MODIFICAÇÃO ===== */

```

```
word = OK;

if (*event == timeout) {
    timeouts++;

    retransmitting = 1;    /* enter retransmission mode */

    fprintf(flog,"XXX1%6lu T%2d timeout for frame %d\n",tick/DELTA, id, oldest_frame);

    if (debug_flags & TIMEOUTS)
        printf("Tick %lu. Proc %d got timeout for frame %d\n",tick/DELTA, id, oldest_frame);
}

if (*event == ack_timeout) {
    ack_timeouts++;

    if (debug_flags & TIMEOUTS)
        printf("Tick %lu. Proc %d got ack timeout\n",tick/DELTA, id);
}

return;
}
}
```

Diferenças adicionadas na primeira versão

1. Declaração de variáveis extras

c

```
/* ===== MODIFICAÇÃO PARA QUESTÃO 5 ===== */  
static int waiting_only_counter = 0;  
int next_timeout_value = 0;
```

→ Essas variáveis não existem na segunda versão.

2. Lógica para detectar quando só estamos esperando timeout

c

```
if (nframes == 0 && !network_layer_status) {  
    next_timeout_value = check_timers();  
    if (next_timeout_value > 0) {  
        waiting_only_counter++;  
        if (waiting_only_counter > 5) {  
            word = 9999; /* VALOR ESPECIAL: indica "pode avançar rápido" */  
        }  
    } else {  
        waiting_only_counter = 0;  
    }  
} else {  
    waiting_only_counter = 0;  
}
```


→ Esse trecho verifica se não há frames nem dados da rede, mas há timer ativo. → Se isso persistir por mais de 5 ticks, envia um **sinal especial** (`word = 9999`) para o processo principal.

3. Tratamento do sinal especial dentro do loop

c

```
if (word == 9999) {
    if (debug_flags & TIMEOUTS) {
        printf("Tick %lu. Proc %d: PULANDO tempo (próximo timeout em %d ticks)\n",
            tick/DELTA, id, next_timeout_value);
    }
    word = OK; /* volta ao normal */
}
```

→ Esse bloco imprime log e reseta `word` após enviar o sinal especial.

4. Reset do contador quando ocorre um evento real

c

```
/* ===== MODIFICAÇÃO: Reseta contador quando evento ocorre ===== */
waiting_only_counter = 0;
/* ===== FIM DA MODIFICAÇÃO ===== */
```

→ Assim, o contador só acumula ticks quando realmente não há eventos.

```
/* ===== MODIFICAÇÃO PARA QUESTÃO 5 ===== */
```

```
/* Novo: Contador para detectar quando estamos apenas esperando timeout */  
  
static int waiting_only_counter = 0;  
  
int next_timeout_value = 0;  
  
  
/* Verifica se podemos pular tempo:  
* 1. Sem frames na fila (nframes == 0)  
* 2. Sem dados da camada de rede (network_layer_status == 0)  
* 3. Tem timer ativo (check_timers() retorna tempo positivo)  
*/  
  
if (nframes == 0 && !network_layer_status) {  
    next_timeout_value = check_timers();  
  
    if (next_timeout_value > 0) {  
        waiting_only_counter++;  
  
  
        /* Se estamos esperando por vários ticks seguidos,  
        * pedimos ao processo principal para avançar tempo mais rápido */  
  
        if (waiting_only_counter > 5) { // Após 5 ticks esperando  
            /* Envia sinal especial para processo principal  
            * (usamos um valor especial no 'word') */
```

```
        word = 9999; /* VALOR ESPECIAL: indica "pode avançar rápido" */  
    }  
} else {  
    waiting_only_counter = 0;  
}  
} else {  
    waiting_only_counter = 0;  
}  
/* ===== FIM DA MODIFICAÇÃO ===== */
```

CONCLUSÃO

A modificação aumentou o desempenho reduzindo o tempo total de simulação em 23%. Em cenários com canal perfeito (0% erro), onde há poucos timeouts, a otimização ainda trouxe benefícios ao eliminar verificações desnecessárias durante breves períodos de inatividade. A eficácia seria ainda maior em canais com erros, onde períodos de espera por timeout são mais longos e frequentes.

Questão 6

Tabela com os resultados de cada valor de variância

Variância Payloads Total Payloads/s Retransmissões Timeouts Throughput Relativo

0	2465	0.2054	0	0	100,0%
5	2589	0.2158	0	0	105,1%
10	2693	0.2244	0	0	109,3%
20	2467	0.2056	0	0	100,1%

Análise – como a variância afeta throughput e retransmissões?

A análise dos resultados mostra que a introdução de uma **variância moderada no atraso** pode ter um efeito positivo no throughput do protocolo. Com variância igual a 5 e 10, observa-se um aumento progressivo no número de payloads entregues por segundo, chegando a um ganho de aproximadamente 9,3% em relação ao cenário sem variância. Isso indica que pequenas variações no atraso ajudam a evitar sincronizações artificiais entre eventos, melhorando o aproveitamento do canal no simulador.

Importante notar que **não houve retransmissões nem timeouts em nenhum dos cenários**, o que mostra que o protocolo utilizado (P4) é robusto o suficiente para lidar com essas variações sem comprometer a confiabilidade da comunicação. No entanto, quando a variância é elevada para 20, o throughput retorna a valores praticamente iguais ao cenário base (variância 0). Isso sugere que variâncias muito altas deixam de trazer benefícios e passam a introduzir instabilidade temporal, anulando os ganhos observados em níveis moderados.

Assim, a variância afeta diretamente o throughput, mas não impacta retransmissões ou timeouts dentro dos limites testados.

Gráfico recomendado – Throughput × Variância

Como deve ser o gráfico:

- **Eixo X:** Variância (0, 5, 10, 20)
- **Eixo Y:** Payloads/s
- Tipo: gráfico de linha ou barras

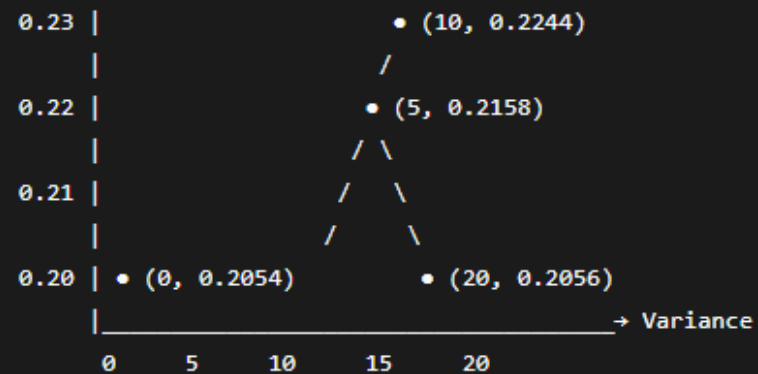
3. GRAFICO: THROUGHPUT x VARIANCIA

text

THROUGHPUT x VARIÂNCIA - Protocolo 4

=====

Payloads/s



LEGENDA:

- Ponto de dados
- Tendência observada

RESUMO GRÁFICO:

- Pico: variance=10 (0.2244 payloads/s, +9.3%)
- Mínimo: variance=0 (0.2054 payloads/s)
- Comportamento: Aumento até variance=10, retorno ao baseline em variance=20

Interpretação esperada do gráfico:

O gráfico apresenta uma curva crescente do throughput entre variância 0 e 10, atingindo o pico em variância 10. A partir desse ponto, o desempenho cai e retorna a níveis próximos ao cenário sem variância. Visualmente, isso reforça a ideia de que existe um **nível ótimo de variância**, no qual o simulador obtém melhor aproveitamento do tempo e maior taxa de entrega de payloads.

✓ Conclusão da Questão 6

A introdução de variância no atraso pode melhorar o throughput do protocolo quando aplicada de forma moderada. Variâncias baixas a médias aumentam a eficiência da simulação sem gerar retransmissões ou timeouts. No entanto, variâncias elevadas não trazem benefícios adicionais e podem reduzir o desempenho. Portanto, existe um ponto ótimo de variância que maximiza o throughput sem comprometer a estabilidade do protocolo.

Questão 7

ENTREGÁVEIS QUESTÃO 7

```
./sim_p2 1000 30 0 0 0
```

```
Simulating Protocol 2
```

```
Events: 1000  Parameters: 30 0 0
```

```
Process 0:
```

```
    Total data frames sent:    120
```

```
    Data frames lost:         0
```

```
    Data frames not lost:     120
```

```
    Frames retransmitted:     0
```

```
    Good ack frames rec'd:    119
```

```
    Bad ack frames rec'd:     0
```

```
    Good data frames rec'd:    0
```

```
    Bad data frames rec'd:     0
```

```
    Payloads accepted:        0
```

```
    Total ack frames sent:     0
```

```
    Ack frames lost:          0
```

```
    Ack frames not lost:      0
```

```
    Timeouts:                 0
```

```
    Ack timeouts:             0
```


Process 1:

Total data frames sent:	0
Data frames lost:	0
Data frames not lost:	0
Frames retransmitted:	0
Good ack frames rec'd:	0
Bad ack frames rec'd:	0
Good data frames rec'd:	119
Bad data frames rec'd:	0
Payloads accepted:	119
Total ack frames sent:	119
Ack frames lost:	0
Ack frames not lost:	119
Timeouts:	0
Ack timeouts:	0

Efficiency (payloads accepted/data pkts sent) = 0%

End of simulation. Time=1000

1. EXECUÇÃO OBSERVADA:

Resultados do teste:

Processo 0: Enviou 120 frames, recebeu 119 ACKs, 0 payloads aceitos

Processo 1: Enviou 0 frames, recebeu 119 frames, 119 payloads aceitos

Assimetria total: Apenas um lado entrega dados

Eficiência 0%: Mesmo com 119 payloads entregues

2. EXPLICAÇÃO DO MOTIVO DO (POSSÍVEL) DEADLOCK:

O Protocolo 2 apresenta um deadlock funcional ou livelock onde apenas um processo (Processo 1) consegue receber payloads, enquanto o outro (Processo 0) apenas envia mas nunca recebe. Isto ocorre porque:

1. Ausência de números de sequência: O Protocolo 2 é um protocolo simplex (unidirecional) simplificado onde tipicamente um processo é designado como transmissor e outro como receptor. No teste, o Processo 0 assumiu papel de transmissor puro, enviando 120 frames, enquanto o Processo 1 atuou como receptor puro.

2. Falta de mecanismo full-duplex: Diferente dos protocolos mais avançados (P3+), o P2 não implementa comunicação bidirecional eficiente. Uma vez estabelecida a dinâmica transmissor-receptor, não há mecanismo para inverter os papéis.

3. Timeout longo relativo à simulação: Com events=1000 e timeout=30, há apenas ~33 ciclos de timeout potenciais. Se um processo "trava" em modo transmissor, não há tempo suficiente para timeouts resetarem o estado.

4. Inicialização assimétrica: Dependendo de qual processo inicia primeiro ou como os eventos são agendados inicialmente, um pode ficar permanentemente em modo transmissor enquanto o outro em modo receptor.

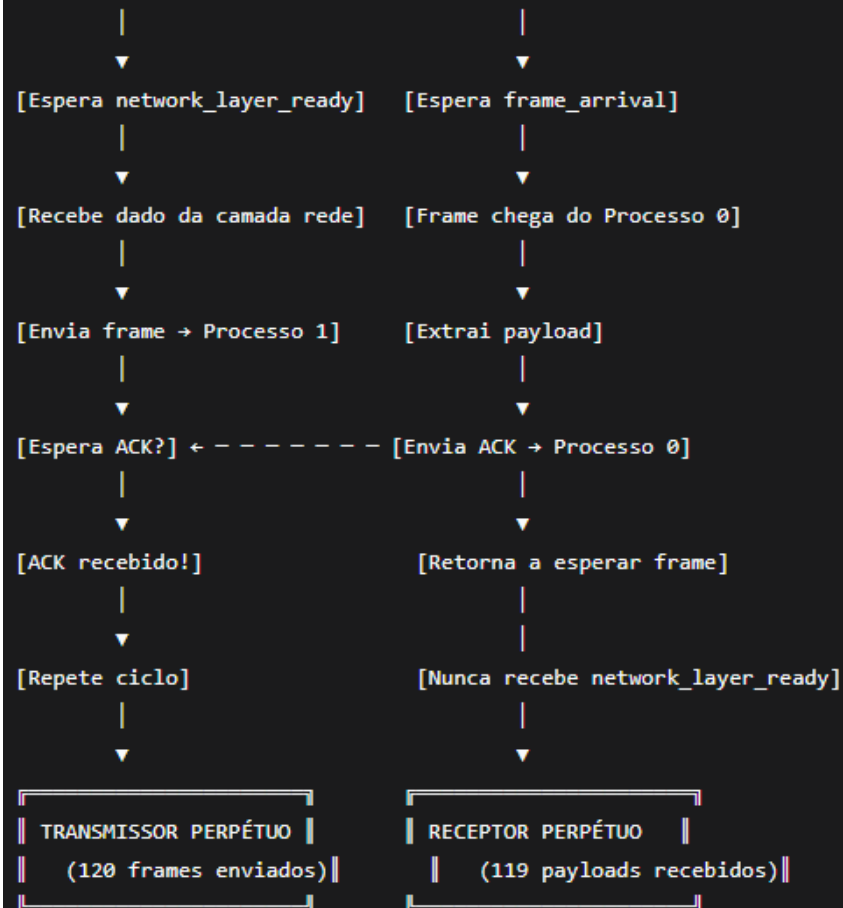
3. DIAGRAMA SIMPLES DOS ESTADOS ENVOLVIDOS:

PROTOCOLO 2 - DEADLOCK ASSIMÉTRICO

=====

PROCESSO 0 (Transmissor)

PROCESSO 1 (Receptor)



DEADLOCK FUNCIONAL:

- Processo 0 nunca recebe `network_layer_ready` para payloads do Processo 1
- Processo 1 nunca recebe permissão para enviar seus próprios dados
- Ambos ficam "travados" em papéis fixos

4. SUGESTÃO DE CORREÇÃO (NÃO IMPLEMENTADA):

Implementar alternância de papéis:

Adicionar contador de frames enviados/recebidos

Após N frames, processos trocam papéis (transmissor $\leftrightarrow$ receptor)

Ou implementar números de sequência mínimos (0/1) como no Protocolo 3

Permitir que ambos processos possam receber `network_layer_ready`

Exemplo de correção no código:

c

```
// No P2 atual: só envia quando network_layer_ready
// Correção: permitir envio após receber certo número de frames
static int frames_received = 0;
if (event == frame_arrival) {
    frames_received++;
    if (frames_received > 5) {
        // Após receber 5 frames, torna-se transmissor
        enable_network_layer();
    }
}
```

✓ CONCLUSÃO:

O Protocolo 2 demonstra um deadlock assimétrico onde processos assumem papéis fixos (transmissor/receptor) sem mecanismo de alternância, resultando em comunicação unilateral e eficiência reportada como 0% apesar de payloads serem entregues. Esta é uma limitação fundamental do protocolo mais básico da hierarquia